Milestone Report: Safety Verification of Chiron IR using Constrained Horn Clauses

Project Overview

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a PDR-based safety verification engine. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Team Information

Roll No.	**Roll No.**	**Email ID**
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in

Source Code

The complete implementation is hosted at: `https://github.com/chiron-verification/milestone-2`
Commit identifier: `cc67c54e062ad6de67d88b2226bb9ebd6e8e79dc`

All verification code resides under `Chiron-Framework/ChironCore/CHC_Verification/`. The pipeline comprises five modules:

- `variable_name_detection_in_IR.py` - scans the IR to build a symbol table of user variables and a counter table of loop-repetition counters.
- `z3_fixed_point.py` - constructs the `Inv` predicate signature and state-variable tuples for the selected mode.
- `init_fixed_point.py` - instantiates the Z3 `Fixedpoint` object (SPACER) and asserts the base-case fact or universally quantified initial rule.
- `step_rules.py` - translates each IR instruction into a universally quantified CHC implication rule.
- `safety_properties.py` - entry point: builds the fixedpoint system, accepts user-specified properties, and reports PASSED/FAILED/UNKNOWN with invariant or counterexample.

Test programs and `unittest` suites (see Tests) reside under `CHC_Verification/tests/`, organised by mode category (`test_specific.py`, `test_default.py`, `test_safety_range.py`, `test_universal/`), covering 19 programs in `programs` folder.

Current Progress

The project began with a thorough literature review and design phase. The implemented pipeline covers the full end-to-end path from a Chiron source program to a property verdict, encompassing state extraction, invariant generation, rule encoding, and property checking.

Literature Review

- **CHCs as a verification IR.** CHC-based encodings yield modular, solver-agnostic verification pipelines [2, 7]; Gurfinkel's notes [3] provide the linear CHC treatment directly applicable here.
- **Z3 Fixedpoint and SPACER.** The μZ engine [5] provides CHC fixed-point computation within Z3; SPACER [1] performs invariant synthesis and reachability queries, returning an inductive invariant or a counterexample.
- **Axiomatic semantics and SeaHorn.** Hoare-style semantics [6] underpin the state-transition encoding; SeaHorn [4] demonstrates the same CHC architecture for C, which our pipeline adapts to Chiron's turtle-graphics semantics.

Implementation Details

Extraction of symbolic program state

File: `CHC_Verification/variable_name_detection_in_IR.py`

The verifier scans the linear Chiron IR to extract all user-declared variables and loop counters introduced by `repeat`, storing each with its corresponding Z3 `Real` variable. Internal repetition counters are kept separate from user variables to avoid name clashes; this determines the arity of the invariant predicate used throughout the pipeline.

Construction of the invariant relation

File: `CHC_Verification/z3_fixed_point.py`

The mode-aware invariant predicate `Inv` captures the full program state: program counter, turtle position `(xcor, ycor)`, heading, pen state, and all user variables and loop counters. Four verification modes are supported:

1. **default**: concrete start state, all values zero, `pendown = False`;
2. **universal**: turtle coordinates, heading, and user variables universally unconstrained;
3. **specific**: user-supplied initial values, with unspecified variables defaulted to zero; and
4. **safety-range**: ghost parameters recording initial user-variable values are appended to `Inv`, enabling the solver's invariant to be read as a precondition over starting values.

Initialization of the CHC system

File: `CHC_Verification/init_fixed_point.py`

The Z3 `Fixedpoint` object (engine: SPACER) is instantiated and seeded with a base-case constraint appropriate to the selected mode: a concrete `fp.fact` for `default`/`specific`, or a universally quantified `fp.rule` for `universal`/`safety-range`.

Translation from Chiron IR to CHC transition rules

File: `CHC_Verification/step_rules.py`

Each IR instruction is mapped to a universally quantified CHC implication over `Inv`:

- **Instruction coverage:** assignments, conditional branching, assertions, `forward`/`backward` movement, `left`/`right` turns, `pendown`/`penup`, `goto`, `pause`, and `NOP`.
- **Expression translation:** arithmetic and boolean expressions are recursively translated to Z3, covering the four arithmetic operations, unary minus, all logical and comparison operators, boolean constants, and pen-status queries.
- **Turtle geometry:** `goto` yields exact coordinate updates; `forward`/`backward` use interval over-approximations of sine and cosine over discretized heading buckets; heading is normalized to $[0, 360)$ via a bounded sequence of conditional rewrites.

Property-checking interface

File: `CHC_Verification/safety_properties.py`

Given a user-provided boolean expression over the state variables, the verifier queries SPACER for a reachable violating state and reports `PASSED`, `FAILED`, or `UNKNOWN`. A counterexample is retained on failure; the synthesised invariant is stored on success. In `safety-range` mode, the invariant is additionally instantiated at the initial state to extract a symbolic safety precondition over starting values.

Usage

The verifier is invoked from `Chiron-Framework/ChironCore/`. The dependecies from Chiron Project suffice to run it. The four supported modes are:

```
uv run python CHC_Verification/safety_properties.py <program.tl> <num_properties> default

uv run python CHC_Verification/safety_properties.py <program.tl> <num_properties> universal

uv run python CHC_Verification/safety_properties.py <program.tl> <num_properties> specific '{"x":
    10, "y": 20}'

uv run python CHC_Verification/safety_properties.py <program.tl> <num_properties> safety-range
```

The tool then prompts interactively for property names and boolean expressions over `xcor`, `ycor`, `heading`, `pendown`, user variables, and loop counters; `And`, `Or`, and `Not` are also available.

Tests

The main evaluation artifacts for this milestone are the automated test modules, the shared test infrastructure, and the `.tl` fixture programs.

Automated test modules

File	Purpose
`tests/test_default.py`	Checks properties in `default` mode, where the entire state starts from a concrete zero initialization. These tests focus on whether the verifier can prove or refute invariants from a fixed start state.
`tests/test_universal.py`	Checks properties in `universal` mode, where user variables and key turtle components are unconstrained initially. These tests validate reasoning under arbitrary initial conditions.
`tests/test_specific.py`	Checks properties in `specific` mode using explicit parameter dictionaries. These tests validate the part of the pipeline that reads concrete input assignments from the command line and injects them into the start state.
`tests/test_safety_range.py`	Checks properties in `safety-range` mode. These tests validate the ghost-parameter mechanism and check whether the implementation can recover a symbolic safe region over initial inputs.
`tests/helpers.py`	Shared testing infrastructure. It builds the fixed-point system for a given program, constructs the property-evaluation context, suppresses verbose pipeline output during testing, and provides helper assertions for expected pass/fail outcomes.
`tests/run_all.py`	Top-level test runner that discovers all `test_*.py` files and executes the complete suite.

Fixture programs used by the tests

Program file	Purpose in evaluation
`assign_basic.tl`	Basic sequential assignments; used to test simple arithmetic invariants and no-motion/no-pen behavior.

`assign_algebra.tl`	Multi-step arithmetic including multiplication and division; used to test algebraic propagation and nonlinear cases.
`conditional.tl`	Simple conditional branch; used to test branching semantics.
`loop_basic.tl`	Counted loop with accumulation; used to test loop invariants.
`loop_accum.tl`	Repetition-driven accumulator growth; used to test monotonic arithmetic invariants.
`loop_cond.tl`	Loop with internal branching and arithmetic updates; used to test interaction between repetition and conditionals.
`goto_computed.tl`	Computed `goto` destinations from symbolic variables; used for geometric safety properties.
`square_goto.tl`	Multiple `goto` commands that trace a square; used to test bounded-region properties.
`forward_square.tl`	Motion using `forward` and `right`; used for trig-dependent geometric tests.
`turns_only.tl`	Pure heading updates; used for directional reasoning and normalization behavior.
`pen_only.tl`	Pen-state updates without arithmetic or motion; used to validate `pendown`/`penup` semantics.
`pen_with_var.tl`	Pen commands combined with variable updates; used to test mixed symbolic and boolean state.
`pen_toggle.tl`	Alternating pen state changes; useful for repeated pen-state transitions.
`param_goto.tl`	Parameterized `goto`; used in `specific` and `safety-range` modes.
`param_loop.tl`	Loop whose initial value depends on a parameter; used to test parameter-sensitive arithmetic invariants.
`param_cond.tl`	Branching behavior controlled by a parameter; used to test mode-dependent initialization and branch selection.
`param_scale.tl`	Arithmetic scaling with an input parameter; used to test concrete parameter injection and sign-sensitive properties.
`param_pen.tl`	Parameterized arithmetic followed by pen commands; used to test the interaction of user inputs and pen state.
`read_before_write.tl`	Reads an input parameter before any local assignment; used to validate handling of externally supplied variables in `specific` mode.

Individual test-case catalogue

The suite contains 89 unit tests in total, grouped into four mode-specific modules. Seven tests are automatically *skipped* at runtime; all involving programs that execute `right` or `forward` commands, producing heading-dependent constraints that are too complex for Spacer to resolve efficiently:

1. **Heading normalization.** Each `right` θ command updates the heading via a chain of nested `If`-expressions that wraps the result into $[0, 360)$. The current encoding produces roughly 20 nested Z3 `If`-nodes per turn, causing the Spacer engine's invariant-inference loop to diverge or time out rather than converge on a stable inductive strengthening.
2. **Trigonometric motion.** `forward` d updates position as $xcor' = xcor + d \cdot \sin(heading)$ and $ycor' = ycor + d \cdot \cos(heading)$. Because SMT solvers do not natively support transcendental functions, the encoding approximates sin and cos with interval over-approximations over discretised heading buckets. These approximations further inflate the constraint size and interact multiplicatively with the heading-normalization chain.

Crucially, the skips reflect a *solver scalability* limitation, not a defect in the encoding logic. The affected programs are correctly translated into CHC rules, and the properties are well-formed; Spacer simply cannot synthesize a sufficiently strong inductive invariant within the configured timeout when both heading normalization and trigonometric approximations are present in the same fixed-point system. Programs that modify heading but do not use `forward`/`backward` (and thus avoid the trig layer) are handled correctly, as demonstrated by the four passed directional tests in default mode and the four active directional tests in safety-range mode. Similarly, programs using `goto` (which sets position directly without trigonometry) pass all geometric tests across all four modes.

Running the tests

This suite requires only the `unittest` module from the Python standard library, other than the dependecies for CHC Verfication and Chiron, which is available in any standard Python 3 installation.

The full automated suite can be run with:

```
cd Chiron-Framework/ChironCore/CHC_Verification/tests
python run_all.py
```

Narrower invocations are also supported:

```
python run_all.py -v                        # verbose
python -m unittest test_default             # one module
python -m unittest test_default.TestDefaultArithmetic.test_arith_assign_z_nonneg_pass
```

Analysis

Strengths of our implementation

- **Modular design.** Variable extraction, invariant construction, initialization, step-rule encoding, and property checking are separated into dedicated components, making the pipeline easy to read, extend, and debug.
- **Multiple verification modes.** The verifier supports four distinct verification questions - concrete default, universal, user-specified, and symbolic safe-range - within a single unified pipeline.
- **Faithful state modeling.** Both program-level variables and turtle-specific state (position, heading, pen) are tracked, allowing the verifier to reason about motion and turtle semantics, not just numeric values.
- **Broad instruction coverage.** The step-rule encoder handles all main Chiron instruction forms: assignments, branching, assertions, movement, turning, pen control, `goto`, and loop counters.
- **Structured test suite.** Tests are organized by mode and property category and include both positive and negative cases, distinguishing provable invariants from reachable violations.
- **Practical CLI workflow.** A user can invoke the verifier on any `.tl` program, select a mode, and interactively enter properties to obtain pass/fail/unknown verdicts with counterexamples or invariants.

Limitations of our implementation

- **Approximate trigonometric reasoning.** Sine and cosine are non-linear, so movement commands are encoded using interval over-approximations over discretized heading buckets. This increases solver burden, reduces precision, and already causes heading-dependent tests to be skipped in the automated suite because the resulting encoding is too complex for `SPACER` to handle efficiently.
- **Heading normalization adds encoding complexity.** Normalizing heading to $[0, 360)$ via a bounded chain of nested `If`-expressions inflates the CHC encoding and makes invariant synthesis harder for the solver, particularly when turns are combined with forward or backward motion.
- **Nonlinear arithmetic causes solver timeouts.** Programs involving multiplication of symbolic quantities generate nonlinear constraints that `SPACER` cannot always resolve, producing `UNKNOWN` instead of a definitive verdict.
- **Test programs are small and pedagogical.** The current benchmark suite covers feature breadth but not depth; no program has been stress-tested with deep nesting, large loop counts, or complex symbolic properties, so scalability remains unvalidated.
- **Properties tested are too simple.** Most verified properties are direct bounds or simple equalities. More realistic safety requirements - combining heading, position, pen state, and user variables across loops and branches - have not yet been formulated or tested.

Data and Visual Analysis

Mode-wise Test Distribution

The suite contains 89 unit tests distributed across the four verification modes. Figure 1 (left) shows the breakdown by mode. Of these 89 tests, 7 are automatically skipped at runtime owing to the higher heading-normalization complexity. Every active test produces the expected verdict, yielding a **100% active-test pass rate** (Figure 1, right).

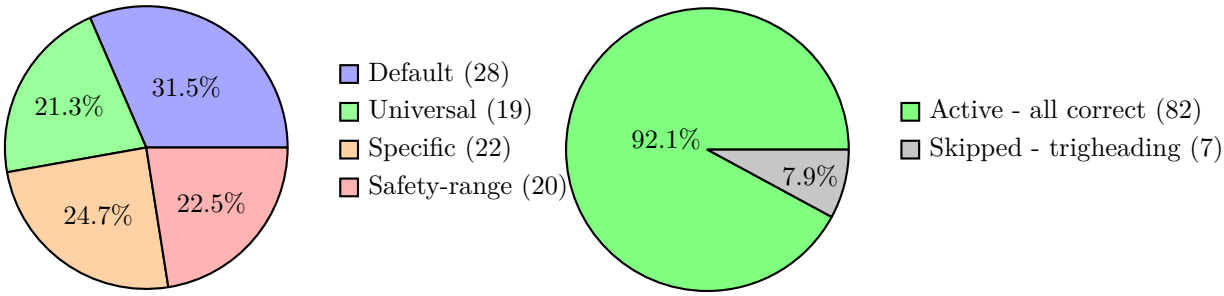

Figure 1: Left: distribution of the 89 tests across the four verification modes. Right: overall test outcomes; every active test produces the expected solver verdict.

Solver Timing Benchmark

Table 3 reports wall-clock times measured by running all test cases through the full pipeline (IR parsing, fixed-point construction, and Z3 SPACER query) on the development machine. Times are split into *build* (parsing + CHC encoding) and *solve* (SPACER query) to isolate solver overhead. (Skipped tests are left blank in the timing columns)

The great majority of cases resolve in well under 200 ms. The single outlier is the FALSE query `x ≤ 30` on `loop_accum.tl` in default mode (1.428 s): SPACER must unroll the accumulator loop enough times to witness a state where $x > 30$, requiring substantially more fixedpoint iterations than a proof query, which terminates as soon as an inductive invariant is found. No query exceeds the 15 s timeout, and there are no UNKNOWN verdicts in this benchmark set.

All these times have been recorded on our local machines, and are expected to vary across different hardware and software environments.

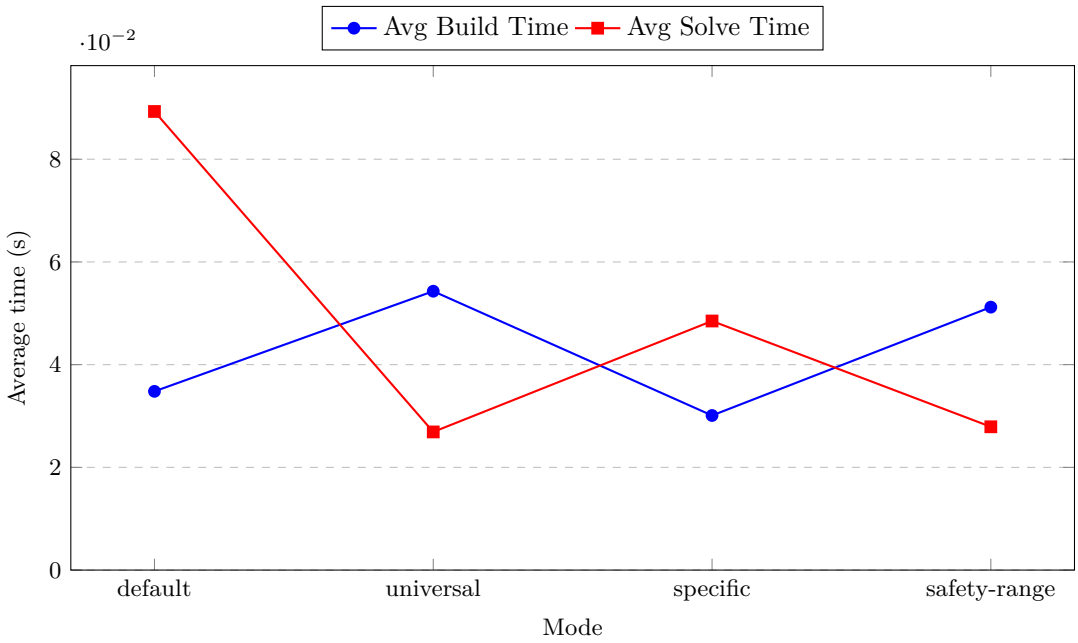

Figure 2: Average build and solve times across verification modes.

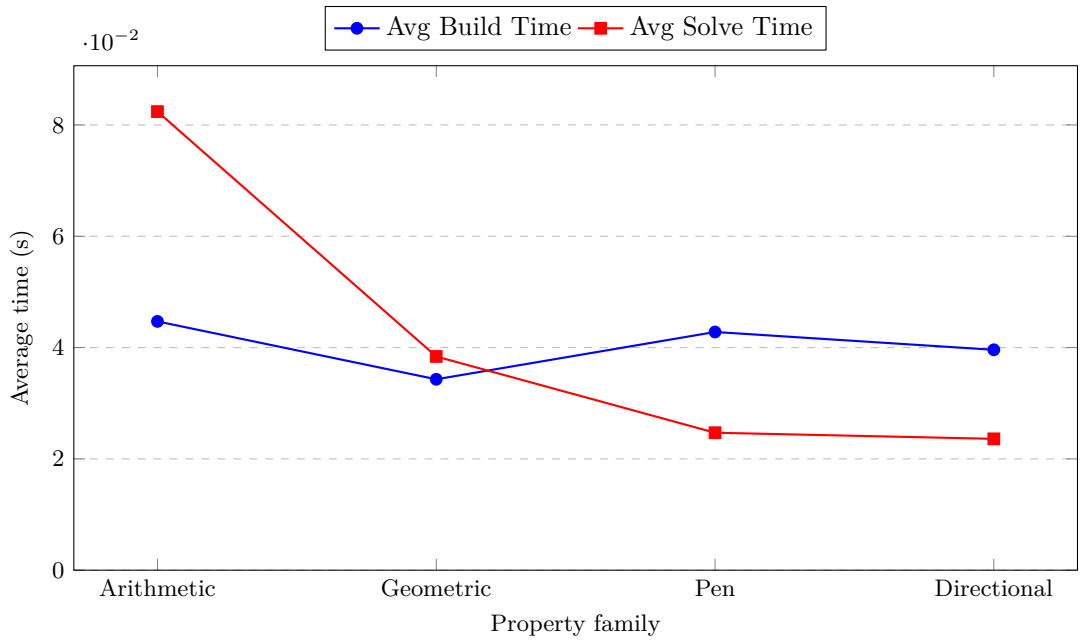

Figure 3: Average build and solve times across property families.

Table 3: Solver timing benchmark.

Program	**Mode**	**Property**	**Verdict**	**Result**	**Build (s)**	**Solve (s)**	**Total (s)**
Default mode - Arithmetic Properties							
`assign_basic`	default	$z \geq 0$	True	Passed	0.020	0.017	0.038
`assign_basic`	default	$z \leq 20$	False	Passed	0.017	0.019	0.036
`loop_accum`	default	$x \geq 0$	True	Passed	0.037	0.008	0.045
`loop_accum`	default	$x \leq 30$	False	Passed	0.037	1.391	**1.428**
`conditional`	default	$z \geq 0$	True	Passed	0.050	0.040	0.090
`loop_basic`	default	$x \geq 0$	True	Passed	0.037	0.018	0.055
`assign_algebra`	default	$b \leq 10$	False	Passed	0.044	0.054	0.098
`loop_cond`	default	$y \leq 100$	True	Passed	0.089	0.205	0.294
`assign_basic`	default	$z \geq 100$	False	Passed	0.021	0.007	0.028
Default mode - Geometric Properties							
`square_goto`	default	bounding box	True	Passed	0.011	0.021	0.032
`square_goto`	default	$xcor \leq 30$	False	Passed	0.012	0.025	0.036
`goto_computed`	default	bounding box	True	Passed	0.025	0.086	0.111
`goto_computed`	default	$ycor \leq 10$	False	Passed	0.025	0.036	0.060
`loop_cond`	default	$xcor = 0$	True	Passed	0.091	0.027	0.118
Default mode - Pen Properties							
`assign_basic`	default	$\neg$ *pendown*	True	Passed	0.020	0.012	0.032
`pen_only`	default	tautology	True	Passed	0.008	0.005	0.013
`pen_only`	default	$\neg$ *pendown*	False	Passed	0.008	0.014	0.022
`loop_basic`	default	$\neg$ *pendown*	True	Passed	0.045	0.027	0.072
`loop_cond`	default	$\neg$ *pendown*	True	Passed	0.095	0.025	0.119
`pen_with_var`	default	$\neg$ *pendown*	False	Passed	0.014	0.019	0.033
Default mode - Directional Properties							
`assign_basic`	default	$heading = 0$	True	Passed	0.019	0.023	0.042
`loop_accum`	default	$heading = 0$	True	Passed	0.038	0.021	0.059
`assign_basic`	default	$heading > 0$	False	Passed	0.017	0.020	0.036
`conditional`	default	$heading = 0$	True	Passed	0.056	0.022	0.078
`turns_only`	default	$heading \geq 0$	True	Skipped	–	–	–

(continued from previous page)

Program	Mode	Property	Verdict	Result	Build (s)	Solve (s)	Total (s)
`turns_only`	default	$heading \leq 90$	False	Skipped	–	–	–
Default mode - Trigonometric Properties							
`forward_square`	default	$xcor, ycor \in [-100, 100]$	True	Skipped	–	–	–
`forward_square`	default	$ycor \geq 0$	False	Skipped	–	–	–
Universal mode - Arithmetic Properties							
`assign_basic`	universal	$z \geq 0$	False	Passed	0.028	0.032	0.059
`assign_basic`	universal	$z{>}0 \vee z{\leq}0$	True	Passed	0.027	0.014	0.041
`loop_basic`	universal	$x \geq 0$	False	Passed	0.057	0.019	0.076
`conditional`	universal	$z \geq 0$	False	Passed	0.078	0.030	0.109
`assign_algebra`	universal	$a \geq 0$	False	Passed	0.042	0.030	0.072
`loop_cond`	universal	$y \leq 100$	False	Passed	0.129	0.033	0.161
`loop_accum`	universal	$x{\geq}0 \vee x{<}0$	True	Passed	0.065	0.032	0.097
`assign_algebra`	universal	$c \geq 0$	False	Passed	0.042	0.026	0.067
Universal mode - Geometric Properties							
`square_goto`	universal	bounding box	False	Passed	0.015	0.022	0.037
`goto_computed`	universal	$xcor \leq 50$	False	Passed	0.033	0.024	0.057
`loop_cond`	universal	$xcor = 0$	False	Passed	0.129	0.028	0.157
Universal mode - Pen Properties							
`assign_basic`	universal	$\neg\, pendown$	True	Passed	0.025	0.015	0.041
`pen_only`	universal	$\neg\, pendown$	False	Passed	0.011	0.024	0.035
`loop_basic`	universal	$\neg\, pendown$	True	Passed	0.055	0.031	0.086
`loop_cond`	universal	$\neg\, pendown$	True	Passed	0.140	0.036	0.176
`pen_with_var`	universal	$\neg\, pendown$	False	Passed	0.019	0.034	0.054
Universal mode - Directional Properties							
`assign_basic`	universal	$heading = 0$	False	Passed	0.026	0.028	0.055
`loop_accum`	universal	$h{\geq}0 \vee h{<}0$	True	Passed	0.056	0.026	0.082
`turns_only`	universal	$heading \geq 0$	False	Skipped	–	–	–
Specific mode - Arithmetic Properties							
`read_before_write`	specific	$z \geq 0$ (`step`=50)	True	Passed	0.018	0.044	0.062
`read_before_write`	specific	$z \leq 100$ (`step`=50)	False	Passed	0.018	0.038	0.056
`read_before_write`	specific	`step` ≥ 0	True	Passed	0.018	0.028	0.046
`read_before_write`	specific	`step` ≤ 30	False	Passed	0.018	0.029	0.047
`param_scale`	specific	$result \geq 0$ (`s`=10)	True	Passed	0.017	0.033	0.050
`param_scale`	specific	$result \leq 30$ (`s`=10)	False	Passed	0.015	0.027	0.043
`param_scale`	specific	$result \geq 0$ (`s`=-5)	False	Passed	0.016	0.040	0.056
`param_loop`	specific	$acc \geq 0$ (`start`=0)	True	Passed	0.071	0.033	0.103
`param_loop`	specific	$acc \leq 20$ (`start`=0)	False	Passed	0.071	0.367	0.439
`param_cond`	specific	$val \geq 0$ (`init`=80)	True	Passed	0.050	0.041	0.091
`param_cond`	specific	$val \leq 60$ (`init`=80)	False	Passed	0.052	0.032	0.084
Specific mode - Geometric Properties							
`param_goto`	specific	box (`sx`=10, `sy`=20)	True	Passed	0.018	0.037	0.055
`param_goto`	specific	$xcor \leq 50$ (`sx`=100)	False	Passed	0.022	0.057	0.079
`param_goto`	specific	$xcor \geq 0$ (`sx`=-50)	False	Passed	0.018	0.031	0.049
`param_goto`	specific	$ycor \leq 50$ (`sy`=10)	True	Passed	0.023	0.038	0.061
Specific mode - Pen Properties							
`param_pen`	specific	$mark \geq 0$ (`b`=5)	True	Passed	0.020	0.032	0.053
`param_pen`	specific	$\neg\, pendown$ (`b`=5)	False	Passed	0.022	0.020	0.043

(continued from previous page)

Program	Mode	Property	Verdict	Result	Build (s)	Solve (s)	Total (s)
`param_pen`	specific	$mark \geq 0$ (`b`=−5)	False	Passed	0.020	0.030	0.051
`param_pen`	specific	$mark \leq 10$ (`b`=3)	True	Passed	0.020	0.025	0.045
Specific mode - Directional Properties							
`param_scale`	specific	$heading = 0$ (`s`=10)	True	Passed	0.015	0.029	0.044
`param_loop`	specific	$heading = 0$ (`s`=5)	True	Passed	0.069	0.029	0.098
`param_cond`	specific	$heading > 0$ (`i`=80)	False	Passed	0.052	0.026	0.078
Safety-range mode - Arithmetic Properties							
`loop_accum`	safety-range	$x \geq 0$	False	Passed	0.045	0.016	0.061
`assign_basic`	safety-range	$z \leq 20$	False	Passed	0.027	0.019	0.046
`assign_basic`	safety-range	$x{\geq}0 \lor x{<}0$	True	Passed	0.028	0.020	0.048
`assign_algebra`	safety-range	$a \leq 10$	False	Passed	0.047	0.019	0.066
`loop_cond`	safety-range	$y \leq 100$	False	Passed	0.123	0.021	0.144
`param_loop`	safety-range	$acc \geq 0$	False	Passed	0.066	0.021	0.087
Safety-range mode - Geometric Properties							
`goto_computed`	safety-range	$xcor \leq 50$	True	Passed	0.033	0.077	0.110
`goto_computed`	safety-range	$xcor \leq 5$	False	Passed	0.032	0.040	0.072
`goto_computed`	safety-range	$ycor \leq 25$	True	Passed	0.032	0.033	0.065
`param_goto`	safety-range	$xcor \leq 50$	False	Passed	0.030	0.033	0.063
Safety-range mode - Pen Properties							
`assign_basic`	safety-range	$\neg\, pendown$	True	Passed	0.041	0.022	0.063
`pen_with_var`	safety-range	$\neg\, pendown$	False	Passed	0.028	0.035	0.063
`loop_cond`	safety-range	$\neg\, pendown$	True	Passed	0.193	0.039	0.231
`param_pen`	safety-range	$\neg\, pendown$	False	Passed	0.029	0.025	0.054
Safety-range mode - Directional Properties							
`assign_basic`	safety-range	$heading = 0$	True	Passed	0.027	0.021	0.049
`loop_accum`	safety-range	$heading = 0$	True	Passed	0.046	0.024	0.069
`loop_accum`	safety-range	$heading > 0$	False	Passed	0.046	0.018	0.064
`assign_algebra`	safety-range	$heading > 0$	False	Passed	0.048	0.020	0.068
`turns_only`	safety-range	$heading \geq 0$	True	Skipped	–	–	–
`turns_only`	safety-range	$heading \leq 90$	False	Skipped	–	–	–

Future Plans

Milestone summary

Milestone 1 Literature review and tooling familiarisation - *completed*: CHC theory studied, Chiron-to-CHC semantics formalised, Z3 Fixedpoint playground implemented.

Milestone 2 End-to-end CHC verification pipeline - *completed*: full IR-to-CHC translation, four verification modes (`default`, `universal`, `specific`, `safety-range`), property-checking interface, CLI, and automated test suite.

Milestone 3 Correctness validation and stress testing - *in progress*: basic regression suite in place; deep-loop and complex-property stress testing, and semantic cross-checks, remain outstanding.

Milestone 4 Final integration, evaluation, and deliverables - *pending*: pipeline consolidation, documentation, final report, and presentation.

Planned work

(a) Pipeline efficiency and trigonometric reasoning

The most pressing technical challenge is the treatment of heading-dependent motion. The current interval-based approximation of sine and cosine over discretized heading buckets is sound but expensive: it inflates the

CHC encoding and already causes solver timeouts on heading-heavy programs. The primary goal for the next milestone is to design a more verification-friendly trigonometric encoding. Reducing nonlinear arithmetic in the encoding more broadly (e.g., replacing symbolic multiplications with bounded linear alternatives where possible) is a secondary priority. If time permits, this phase will also include exploration of additional safety property classes.

(b) Canvas-marking safety properties

A distinctive class of safety properties for Chiron concerns the *canvas*: specifically, which regions of the 2D plane will become marked (pen down during motion), and which are guaranteed to remain unmarked. We plan to implement and verify properties of the form: "the turtle never draws in region R", "all drawing is confined to a bounding box", and "the pen is always up when the turtle is in region S". These require reasoning jointly about turtle position, heading, and pen state across all reachable program states, and will constitute a meaningful stress test for the pipeline.

(c) Testing, stress testing, and end-to-end integration

The final milestone centres on robustness and completeness. Planned work includes:

- extending the benchmark suite with programs involving deep nesting, large loop counts, compound arithmetic, and combined geometric and variable-level behavior;
- testing with complex, multi-component safety properties involving both turtle state and user variables;
- reducing the fraction of `UNKNOWN` verdicts through encoding-level improvements and solver parameter tuning; and
- consolidating the full pipeline into a clean, reproducible end-to-end tool with stable CLI outputs, a finalized test suite, and complete documentation.

Declaration

We declare that the work presented in this report is our own and has been carried out in accordance with the guidelines set out by the course. We have not copied any part of this report from any other source, and we have not submitted this report to any other course or institution for assessment. We understand that any violation of this declaration may result in disciplinary action.

References

[1] Generalized pdr with spacer. Online Z3 Guide, 2026. Available at: `https://microsoft.github.io/z3guide/docs/fixedpoints/engineforpdr/`.

[2] Nikolaj Bjørner, Arie Gurfinkel, Kenneth L. McMillan, and Andrey Rybalchenko. Horn clause solvers for program verification. In *Fields of Logic and Computation II*, pages 24–51. Springer, 2015.

[3] Arie Gurfinkel. Constrained horn clauses (chc). ECE 750T Lecture Notes, University of Waterloo, 2018.

[4] Arie Gurfinkel, Temesghen Kahsai, Anvesh Komuravelli, and Jorge A. Navas. The seahorn verification framework. In *Computer Aided Verification (CAV)*, 2015.

[5] Kryštof Hoder, Nikolaj Bjørner, and Leonardo de Moura. μz– an efficient engine for fixed points with constraints. In Ganesh Gopalakrishnan and Shaz Qadeer, editors, *Computer Aided Verification*, pages 457–462, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[6] University of Texas at Austin. Axiomatic semantics and hoare logic. Course Lecture Notes.

[7] Andrey Rybalchenko. Constraint solving for software verification. In *VMCAI*, 2011.